

Space Robotics – Assignment 3

Lucas Plumbohm

...

Elijah Spannenberg

...

Mathis Beissner

26665176

1 Generative AI Disclosure

Generative AI tools were used to assist with analysing error messages during the debugging process, creating helper scripts for advanced visual debugging, formatting this report as a $\text{\LaTeX}$ document, and correcting spelling and wording throughout the report.

2 Statement of Individual Contributions

Task	Completed?	Team member	Contribution
Perception 1	Yes	Mathis	All
Perception 2			
Perception 3			
Planning 1			
Planning 2			
Planning 3			
Advanced 1			
Advanced 2			
Advanced 3			
Advanced 4			
Advanced 5			
Advanced 6			
Report writing			
Recorded videos			

3 Project Overview

TBA

4 Implementation

TBA

4.1 Perception 1: Collect an Image Dataset of the Artefacts

The goal of this task was to collect images of the artefacts to create a dataset for the subsequent object detection task. The first step was to implement a manual capture mechanism so that specific, relevant scenes could be selected and saved. This was achieved by adding a ROS 2 subscriber that listens for `std_msgs/msg/Empty` messages. Publishing one of these messages manually triggers the `capture_callback` method shown below, which sets the `capture_requested_` flag to `True` and logs the request.

The system receives new camera images every 0.2 seconds. When a capture has been requested, the image callback saves the image data and then resets the request flag. This allows individual scenes to be saved without storing every frame from the camera stream.

Since depth information could also be useful for the later detection tasks, the existing image processing was extended by adding a depth image subscriber. The RGB and depth subscribers were synchronised so that a combined callback is triggered when corresponding images are available. In addition to the depth data, a depth image suitable for visual inspection can be saved. A new configuration file, `detection_config.yaml`, was added to specify the output directories and which image outputs should be saved.

```
1 def capture_callback(self, msg):  
2     self.capture_requested_ = True  
3     self.get_logger().info('Requesting image capture.')
```

Listing 1: Implementation of capture callback



Figure 1: Example image from the collected dataset.

- 4.2 Perception 2: Detect Artefacts with Computer Vision
- 4.3 Perception 3: Artefact Localisation and Display
- 4.4 Planning 1: Autonomously Explore the Cave
- 4.5 Planning 2: Close-Range Inspection
- 4.6 Planning 3: Behaviour Switching
- 4.7 Advanced 1: Robust Perception
- 4.8 Advanced 2: Cave Geometry Analysis
- 4.9 Advanced 3: Communication Network Deployment
- 4.10 Advanced 4: Online Roadmap Construction
- 4.11 Advanced 5: Science Tour Planning
- 4.12 Advanced 6: Adaptive Scientific Sampling
- 5 Results
- 6 Reflection on Teamwork